Lesson 7 Python Container Type and Related Operation

1. Container Introduction

Object is the abstract of data in Python. All the data in Python program is represented by object or the relationship between objects. A Container is an object that contains the references to other objects, such as Tuple, List, Dictionary, etc. These referenced objects compose the object value of the container.

There are two common types of containers, including sequence (List, Tuple, etc.) and map (Dictionary). In sequence, each element has subscript and they are organized. In map, each element has its name (key), and they are unordered. Apart from sequence and map, "**Set**" is also a container.

2. Container Introduction

2.1 List

1) Syntax: [123 , "456" , classObjectA , [123,"456"]]

2) Format: list is surrounded by square bracket and its elements are separated by comma. The type of the elements is diverse, and they can be nested, and read and written by subscript.

3) Feature: There is no unified type of the elements in List, and they can be added, deleted, modified and searched. We can delete (pop) the elements based on subscript, remove elements based on value, insert elements into some position and append the elements to the end of List.

```
list1=[123,"123",[123,456]]
```

2.2 Tuple

1) Syntax：（123，456，789，100）。Syntax: (123，456，789，100)

2) Format: surrounded by bracket and the elements are separated by comma. The elements can be nested, but the type of elements must be unified. They can be read by subscript, but cannot be written, added or deleted.

3) Feature: it is considered as immutable list and all the operations on it are similar to List. The only difference is that Tuple can only be searched. It cannot be added(append and insert), delete(remove and pop) and modify (assign value to elements). In other words, Tuple is a list whose elements should be unified and immutable, and content can only be read.

```
tuple1=(123,456,789)
```

2.3 Set

1) Syntax: { 1，"23"，classObjectA }

2) Format: it is surrounded by curly bracket. Its elements are separated by comma. The type of the elements is diverse, and they can be nested, and read and written by subscript

3) Feature: apart from automatic deduplication and not supporting sorting, others are the same as list.

```
set1={123,"123"}
```

2.4 Dictionary

1) Syntax: {"name" ："zhangsan"， "age"：50}

2) Format: it is surrounded by curly bracket and the elements are separated by comma. The elements should be in unified format (When the elements are separated by semicolon, the front part should be string and the latter part can be any types.) and Key must be string. If there is no semicolon within some element, dictionary will be considered as Set.

3) Feature:dictionary is the most flexible container. It doesn't support sorting, but support nesting. Access the elements through [ ] and get and the elements cannot use subscript. Delete the elements by pop key. (without remove)

```
dict1={"123":123,"456":"abc",{"qwe":789}}
```

3. Basic Operation of Container

3.1 Basic Operation of List

Type	Keyword/ function/ method	Meaning
Add	append()	Append elements to the end
	insert()	Insert the data into the designated position
Delete	clear()	Clear the list
	pop()	Pop up the data at the end by default
	pop(index)	Pop up the designated index data

	remove (data)	Remove the designated data
Modify	extend(list 2)	Add the data of list 2 to the end of the current list
Check	count(data)	Count the frequency of occurrence of the data
	index(content)	Check the position of the content
Other	copy()	Copy a piece of list
	sort()	sort
	reverse()	Reverse the list

1) append：Add the elements to the end of the list

2) insert：Insert the elements into the designated position.

3) pop：Pop up the last element. And you can also specify the position of the pop-up element

4) remove：Delete the designated elements

```
list1=[1,2,3,4,5,6,7]
list1.append(10)
print(list1)
list1.insert(2,3)
print(list1)
list1.pop(0)
print(list1)
list1.remove(7)
print(list1)
```

```
[1, 2, 3, 4, 5, 6, 7, 10]
[1, 2, 3, 3, 4, 5, 6, 7, 10]
[2, 3, 3, 4, 5, 6, 7, 10]
[2, 3, 3, 4, 5, 6, 10]
```

5) index：Check the position of the designated elements

6) sort：Sort the content in ascending order, or in descending order.

```
list1=[1,7,3,8,5,9,4]
print(list1.index(4))
list1.sort()
print(list1)
```

```
6
[1, 3, 4, 5, 7, 8, 9]
```

3.2 Basic Operation of Tuple

The operations on tuple are similar to those on list, but they are still different.

1) Tuple doesn't have append(), extend(), insert(), etc., therefore we cannot add element to the tuple.

2) remove() and pop() are also not contained, hence we cannot delete elements from the tuple.

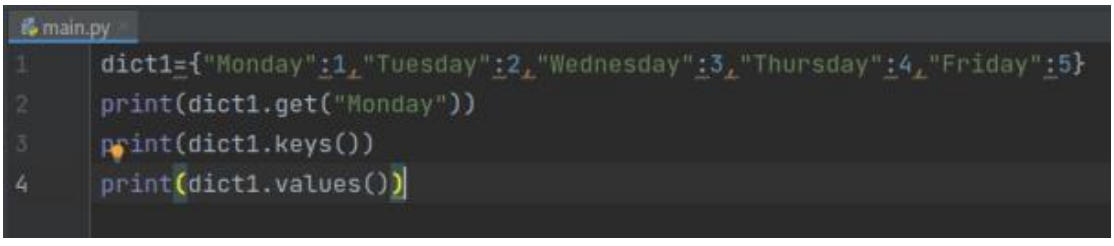
3.3 Basic Operation of Dictionary

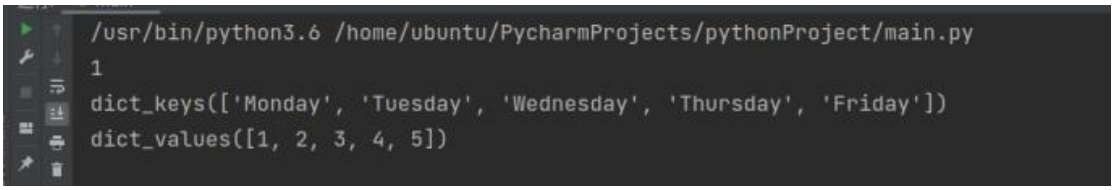
Method	Usage
get (key, default=None)	Return the value of designated keys. If the value cannot be find in the dictionary, default value will be returned.
keys()	Return all the keys of a dictionary as list
values()	Return all the values of the dictionary as list
update(dict2)	Update the key or value of dictionary dict to dict
items()	Return tuple data, keys or value, that can be traversed

3) get: Return the value of designated key

4) keys: Return all the keys of the dictionary

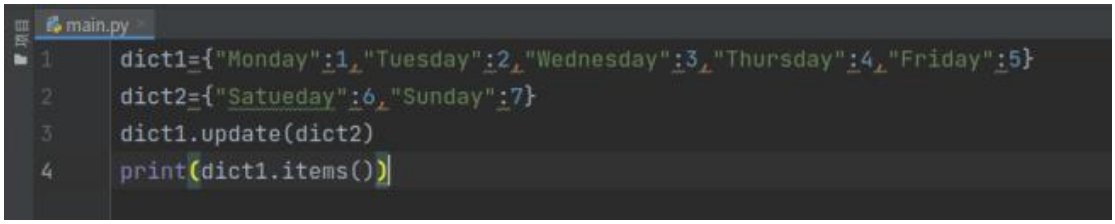
5) values：Return all the values of the dictionary





6) Update：Update the existing key-value pairs or add new key-value pairs

7) items：Return an array of key-value tuples that can be traversed as a list




```
/usr/bin/python3.6 /home/ubuntu/PycharmProjects/pythonProject/main.py
dict_items([('Monday', 1), ('Tuesday', 2), ('Wednesday', 3), ('Thursday', 4), ('Friday', 5), ('Satueday', 6), ('Sunday', 7)])
```

Lesson 8 Python Class and File Operation

1. Object Oriented Programming

Python is object-oriented advanced language where everything is considered as object. Object-oriented programming is to bound variable with data type, then classify and encapsulate so as to reduce repetitive codes. Object focuses on the encapsulation of functions and variables. However object-oriented focuses on who takes charge of encapsulation, and encapsulate multiple methods in an object based on the operator.

Two core definitions: classes and objects

2. Class and Object Introduction

2.1 Class

Class is used to describes the object sets with the same properties and methods. And it defines the properties and methods shared by all the objects of this set. Object is the instance of class.

For example, create a “cat” class, and define the name, color, age, etc., as well as the way to eat, drink, sound, etc. And instantiated object “ginger cat” will have these defined properties and methods.

2.2 Object

An Object is an instance of a Class, which holds the properties and methods of class. In Python, all data types are considered as object, and objects can be

user-defined. User-defined object data type is equal to class in object-oriented.

3. Class Definition

1) Defined by class keywords, class name usually starts with capital letter.

2) _init_ function initializes class, which is equal to constructor. Whenever creating instance based on class, this function will be ran automatically in Python.

3) self function, implicit parameter and the object itself, is equivalent to this function.

4) Properties correspond to member variables.

```
1  class Cat():
2      def __init__(self,name,age):
3          self.name=name
4          self.age=age
5      def eat(self):
6          print("Let's Eat!")
7      def meow(self):
8          print("MEOW~")
```

4. Instance Creating

1) Create the class name, and input the required properties.

2) These properties and methods can be called directly.

```
my_cat=Cat("Tom","6")
print("Name:"+my_cat.name+" Age:"+my_cat.age)
my_cat.eat()
my_cat.meow()
```

```
Name:Tom Age:6
Let's Eat!
MEOW~
```

5. Property of Class and Instance

1) Instance property are defined within **__init__** of a class, and class property are defined outside __init__ of a class. An instance of a class created outside the class can define its own properties.

2) Personal variable: if don't want the external to access some properties, we can start the property name with double subscripts (__). Therefore, these properties will not be accessed by original variable name, that is, they can not be called from an instance.

6. Class Inheritance and Polymorphism

1) Inheritance: when creating a class, you can write the name of the parent class in parentheses, so that the properties and methods of the parent class are inherited to the child class. If the child class does not have an initialization method, the initialization method of the parent class will be called.

2) Method rewriting: If the name, return type and parameter list of a method defined in a child class exactly match those of a method in the parent class, the method of child class rewrites that of parent class.

3) Polymorphism: the behaviors of multiple child classes derived from one parent class are different, which is called polymorphism. In other words, child class rewrite the methods of parent class to let it has methods different from the parent class. When child class have the same method as parent class, the priority will be given to the method of child class, that is, the child class overrides the parent class.

```
class Animal():
    def __init__(self,name,age):
        self.name=name
        self.age=age
    def eat(self):
        print("Let's Eat!")
class Cat(Animal):
    def eat(self):
        print("Eat Fish!")
class Mouse(Animal):
    def eat(self):
        print("Eat Cheese")
my_cat=Cat("Tom","6")
print("Name:"+my_cat.name+" Age:"+my_cat.age)
my_cat.eat()
my_mouse=Mouse("Jerry","5")
print("Name:"+my_mouse.name+" Age:"+my_mouse.age)
my_mouse.eat()
```

```
Name:Tom Age:6
Eat Fish!
Name:Jerry Age:5
Eat Cheese!
```

After execution, the instance of child class will automatically call the construction method of parent class, and the child class will override the parent class method with the same name.

7. File Operation

7.1 File and File Path

1) Two key properties: “file name” and “path”

2) Absolute path and relative path: “**Absolute Path**” always starts with root folder. “**Relative Path**” corresponds to the current working directory of the program. For example, the relative path of “**test.py**” file under the current working directory is “**/test.py**” and its absolute path is “**python/scripts/test.py**”.

3) (.) and (..) folder: Not real folders, they are the special name that can

be used in the path. When a single period (".") is used as a folder directory name, it is an abbreviation for this directory. Two periods ("..") mean the parent folder.

7.2 File Reading Operation

1) Open and close file: call open function to pass in file name and identifier. Open file to return a file object, and then call close() method of File object to close this file. “with” statement can also be used to close the file automatically when ending.

2) File reading: call read function to read the file content to the object. Use readline function to read one line at a time or read all lines at a time, and save them in one list.

3) File writing：When opening a file, you can add designation parameters: read mode ("r"), write mode ("w") [write from the beginning], append mode ("a") [write at the end of the file], read at the same time and write the mode of file ("r+"). You can also write directly.

```
filename = 'num.txt'
with open(filename,'r+') as file_object:
    file_object.write("Hiwonder")
file_object=open(filename,'r')
lines = file_object.readlines()
print(lines[0])
```

```
Hiwonder
```

After execution, the first line of data just written will be read.

Lesson 10 Python Built-in Function

1.Built-in Function Definition

Python compiler is a program that provides users with common functions with unique names. And these common functions are built-in functions.

Built-in function is one part of compiler and takes effect as compiler starts. Standard library function is the external expansion of the compiler, and takes effect after the module is imported. In general, the execution efficiency of built-in function is higher than that of standard library function.

The amount of built-in functions must be strictly controlled, otherwise Python compiler will be “overstaffed”. Generally speaking, only the functions that are frequently used and are tightly bound to the language itself will be promoted as built-in function.

2. Built-in Function List

abs()	delattr()	hash()	memoryview()	set()
all()	dict()	help()	min()	setattr()
any()	dir()	hex()	next()	slicea()
ascii()	divmod()	id()	object()	sorted()
bin()	enumerate()	input()	oct()	staticmethod()
bool()	eval()	int()	open()	str()
breakpoint()	exec()	isinstance()	ord()	sum()
bytearray()	filter()	issubclass()	pow()	super()
bytes()	float()	iter()	print()	tuple()
callable()	format()	len()	property()	type()
chr()	frozenset()	list()	range()	vars()
classmethod()	getattr()	locals()	repr()	zip()
compile()	globals()	map()	reversed()	__import__()
complex()	hasattr()	max()	round()	

2.1 Operation Type

1) abs(x)：abs function is used to return the absolute value of a value. The input x parameter can be the floating point number of the integer, and also can be complex number.

2) max(x，key=None)：The parameter x of the max function is an iterable object or multiple parameters, and max function will return the maximum element among them. max function can return the maximum value through designating the key parameter. If there are multiple maximum values, the first value will be returned.

3) min(x,key=None)：The usage of min function is the same as that of

max function. min function will return the minimum value of an iterable object or multiple parameters.

```
print(abs(-46))
print(max(2,3,4,5))
print(min(2,3,4,5))
```

```
46
5
2
```

4) pow(x,y, mod): When pow function only has x and y parameters, its function is to return x to the y power. When the third parameter mod exits, the pow function takes the remainder of mod on the basis of x raised to the y power.

5) round(number,ndigits)：Return the value of number rounded to ndigits of precision after the decimal point. If ndigits is omitted, the integer closest to the number is returned.

6) sum(x, start=0): sum function is to sum the values in x from left to right. Then add the value of start, sum and return the total value.

7) divmod(a, b): divmod function takes a and b as arguments, and return the quotient and remainder of a/b.

```
print(pow(2,3))
print(round(1.2345,3))
print(sum((1,2,3,4),2))
print(divmod(5,2))
```

```
8
1.234
12
(2, 1)
```

2.2 Convert the Type

1) int(x): If x is a number or string, the integer of x will be returned. If x is empty, 0 will be returned.

2) float(x): If x is a number or string, the floating point number of x will be returned. If x is empty, 0 will be returned.

3) complex(real, imag): It will return the complex form of **real + imag*1j**, or converts a string or number to the complex form. If the first parameter “real” is a string, the second parameter “**imag**” should be omitted.

```
print(int(2.3))
print(float(3))
print(complex(2,3))
```

```
2
3.0
(2+3j)
```

4) bool(x): Determine parameter x is true or false, and return **True** or **False**.

5) str(x): Convert the input x to string and return the result.

6) bytearray(x, encoding="utf-8"): bytearray converts the input x to ordered and mutable array composed of bytes, and return the array.

7) bytes(x, encoding="utf-8"): bytes convert the passed in parameter x to an immutable array of bytes, and return this array.

8) memoryview(x): it returns the memory view object of the parameter x, and the memory view object allows the data that supports the buffer protocol to be packaged, and the returned object is a list of tuples. Note: the incoming parameter x must be bytes.

```
# bool function
print(bool(3>4))
print(bool("2"=="2"))
# str function
print(str(3.45))
print(str([3,4]))
# bytearray function
print(bytearray("python", encoding="utf-8"))
# bytes function
print(bytes("python", encoding="utf-8"))
# memoryview function
print(memoryview(bytes(123)))
```

```
/usr/bin/python3.8 /home/lin/PycharmProjects/pythonProject/test.py
False
True
3.45
[3, 4]
bytearray(b'python')
b'python'
<memory at 0x7f554c5eba00>
```

2.3 Base Conversion

1) bin(x): Convert the input integer x to a binary string prefixed with "0b"

2) oct(x): Convert the input integer x to an octal string prefixed with "0o"

3) hex(x): Convert the input integer x to a hexadecimal string prefixed with "0x".

4) ord(x): For the input single Unicode character, its corresponding Unicode integer will be returned.

5) chr(x): The chr function is the inverse of the ord function, and is used to return the corresponding single Unicode character for the input integer x.

```
# bin function
print(bin(3))
# oct function
print(oct(3))
# hex function
print(hex(3))
# ord function
print(ord('c'))
# chr function
print(chr(99))
```

```
/usr/bin/python3.8 /home/lin/PycharmProjects/pythonProject/test.py
0b11
0o3
0x3
99
c
```

2.4 Numerical Operations

1) list(x): list function changes the incoming parameters to a new list and returns it. In addition, the list itself is also a mutable object.

2) dict(x): dict function changes the incoming parameter x into a new dictionary object and returns it. The dictionary object is immutable.

3) set(x): set function is used to change the input parameter x to a new collection object and return it. The set object is mutable, and its internal elements are not repeatable.

4) frozenset(x): frozenset function can also complete the function of the set function, but the frozenset object is immutable. Therefore it is not possible to insert values into the frozenset object.

```
# list function
print(list("1234"))
print(list())
# dict function
print(dict(a=2,b=4))
print(dict())
# set function
print(set())
set1 = set([1,2,3,4,1,2,3,4])
print(set1)
set1.add(5)
print(set1)
# frozenset function
frozenset1 = frozenset([1,2,3,4,1,2,3,41])
print(frozenset1)
```

```
/usr/bin/python3.8 /home/lin/PycharmProjects/pythonProject/test.py
['1', '2', '3', '4']
[]
{'a': 2, 'b': 4}
{}
set()
{1, 2, 3, 4}
{1, 2, 3, 4, 5}
frozenset({1, 2, 3, 4, 41})
```

5) tuple(x): tuple function changes the incoming parameter x into a new tuple object and returns it. The tuple object is immutable.

6) enumerate(x): It returns an enumeration object, and the input parameter x is an iterable object. Through **__next__()**, the returned enumeration object returns a tuple containing the count value and the value in x obtained by iteration.

7) range(x): The range function generates an immutable sequence of numbers from an input x, typically used to loop a specified number of times in a **for loop**

8) iter(x): The iter function generates an iterable object based on the input parameter x and returns the iterable object.

```
# tuple function
print(tuple([1,2,3,4]))
print(tuple())
# enumerate function
for index, value in enumerate("abed"):
    print(index, ":", value)
# range function
print(range(10))
print(range(0,10,2))
# iter function
iter1= iter([1,2,4,5])
for i in range(3):
    print(next(iter1))
```

```
/usr/bin/python3.8 /home/lin/PycharmProjects/pythonProject/test.py
(1, 2, 3, 4)
()
0 : a
1 : b
2 : e
3 : d
range(0, 10)
range(0, 10, 2)
1
2
4

进程已结束,退出代码0
```

9) slice(x): The slice function is mainly used as flexible constructed slices, and it also returns a slice object.

10)object(): In Python, the object class is the base of all classes. The object function does not accept any parameters and returns an object without any characteristics.

11)super(): The super function is often used in Python class objects. In the inheritance of child classes from the parent class, the child class is used to refer to the objects and methods in the parent class without explicitly

specifying the name of the parent class.

```
# slice function
print(slice(5))
print(slice(0,5,2))
# object function
print(object())

# super function
class A:
    def __init__(self):
        self.fathername = "A"
class B(A):
    def __init__(self):
        super().__init__()
        self.sonmane = "B"
print(B().fathername)
print(B().sonmane)
```

```
/usr/bin/python3.8 /home/lin/PycharmProjects/pythonProject/test.py
slice(None, 5, None)
slice(0, 5, 2)
<object object at 0x7ff022f36d60>
A
B
```

2.5 Sorting Operation

1) sorted(x, key=None, reverse=False): Sort the iterable object x and return a new sorted object. The key parameter can specify how to compare, and when reverse is True, it indicates sorting in descending order.

2) reversed(seq): Reverse the input sequence seq to generate a new iterable object and return.

```
# sorted function
print(sorted([1,3,6,4,7,2]))
print(sorted([1,3,6,4,7,2], reverse=True))
print(sorted({"a":3,"b":6,"c":11}.items(), key=lambda x: x[1]))
# rever function
print(list(reversed([1,2,3,4,5,6])))
print(list(reversed("abcd")))
```

2.6. Sequence Operation

1) all(x): Determine whether each element of the iterable object x is true, and if one element is False (0), return False.

2) any(x): Same as the all function, the any function determines whether each element in x(iterable object) is True, and returns True as long as one element is True.

3) map(func, iter): The map function returns an iterator in which the func function will be applied to each element in the iterable object iter

4) filter(func, iter): Filter the element values in the iterable object iter through the func function, and return a new iterator composed of the filtered elements.

5) next(iter): The next function returns the next element in the iterable object.

```
# all function
print(all([i>2 for i in [1,2,3,4,5]]))
# any function
print(any([i>2 for i in [1,2,3,4,5]]))
# map function
def func(x):
    return x > 3
print(map(func,[1,2,3,4,5]))
for i in map(func,[1,2,3,4,5]):
    print(i,end=')t')
# filter function
print(filter(func,[1,2,3,4,5]))
for i in filter(func,[1,2,3,4,5]):
    print(i,end='It')
# next function
iterl =iter([1,2,3])
for i in range(3):
    print(next(iterl), end='It')
```

```
/usr/bin/python3.8 /home/lin/PycharmProjects/pythonProject/test.py
False
True
<map object at 0x7f0d38ddd6d0>
False )tFalse )tFalse )tTrue )tTrue )t<filter object at 0x7f0d38ddd6d0>
4 It5 It1 It2 It3 It
```

6) zip(*iter): According to multiple different iterators, the aggregation of elements in corresponding position is performed, and a new iterator is returned.

```
# zip function
for a,b in zip([1,2,3,4,5],"abcde"):
    print(a,b)
```

```
/usr/bin/python3.8 /home/lin/PycharmProjects/pythonProject/test.p
1 a
2 b
3 c
4 d
5 e
```

7) reduce(func,sequence[,initial]): Python3.x中需要从functools中导入reduce函数再使用。The two-argument function func is applied iteratively to each element in the sequence seq from left to right, and finally returns a single value as the result. It is a built-in function in Python2.x, and in Python3.x, you need to import the reduce function from functools first.

```
import functools
def add(x,y):
    #Add two numbers
    return x+y
suml = functools.reduce(add,[1,2,3,4,5])#Sum the lists:1+2+3+4+5
sum2 = functools.reduce(lambda x,y:x+y,[1,2,3,4,5])
print(suml)
print(sum2)
```

```
/usr/bin/python3.8 /home/lin/PycharmProjects/pythonProject/test.py
15
15
```

2.7. Object Element Operation

1) help(object): The help function can help users to query the information of different objects, including built-in methods, properties and other information.

2) id(object): Returns the identity value of the object, which is an integer and remains unique in the object's life cycle.

```
# help function
help(int)
```

```
/usr/bin/python3.8 /home/lin/PycharmProjects/pythonProject/test.py
Help on class int in module builtins:

class int(object)
 |  int([x]) -> integer
 |  int(x, base=10) -> integer
```

```
# id function
num = 10
print(id(num))
```

3) hash(object): If the object has a corresponding hash value, the corresponding hash value is returned.

4) type(object): The type function is used to return the type of the object.

5) dir(object): If there is no argument object, the dir function returns a list of names in the current local scope. If there is an argument object, the function attempts to return a list of valid properties for that object

6) len(object): Return the length of the object or the number of elements it contains.

```
# hash function
print(hash("python"))
# type function
str1 = "python"
list1 = [1,2]
print(type(str1))
print(type(list1))
# dir function
print(dir())
# len function
list2 = [1,2,3,4]
str2 = "ererdfff"
print(len(list2))
print(len(str2))
```

```
/usr/bin/python3.8 /home/lin/PycharmProjects/pythonProject/test.py
-2607102823527459755
<class 'str'>
<class 'list'>
['__annotations__', '__builtins__', '__cached__', '__doc__', '__file__', '__loader__', '__name__', '__package__', '__spec__', 'list1', 'str1']
4
8
```

7) repr(object): The repr function returns a string containing a printable representation of an object.

8) ascii(object): The ascii function is similar to the repr function, the ascii function returns a string containing a printable representation of an object. However unlike the repr function, ascii() escape non-ASCII encoded characters.

9) format(value ,format_spec): format function converts value into a "formatted" representation controlled by the format_spec parameter, and is mostly used in string formatting.

10)vars(object): The function returns objects with a **_dict__** attribute, such as modules, classes, instances, etc.

```
# repr function
print(repr([1,2,3,4]))
print(repr("python"))
# ascli function
print(ascii([1,2,3,4]))
print(ascii("python"))
print("python ()".format(123))
#  Output in octal 123
print(format(123,'o'))
#Output in hexadecimal
print(format(123,'x'))
# vars function
class A:
    def __init__(self):
        self.name = "python"
        self.age =10
print(vars(A()))
```

```
/usr/bin/python3.8 /home/lin/PycharmProjects/pythonProject/test.py
[1, 2, 3, 4]
'python'
[1, 2, 3, 4]
'python'
python ()
173
7b
{}
```

2.8. Property Operation

1) isinstance(object, classinfo): The function is used to determine whether the object belongs to the type of classinfo, if it is, it returns True, otherwise it returns False

2) issubclass(class, classinfo): If class is a child class of classinfo class, the function returns True, otherwise it returns False.

```
# isinstance function
print(isinstance("python", str))
print(isinstance("python", int))
# issubclassa function
class A:
    def __init__(self):
        pass
class B:
    def __init__(self):
        pass
print(issubclass(B,A))
print(issubclass(int,object))
```

```
/usr/bin/python3.8 /home/lin/PycharmProjects/pythonProject/test.py
True
False
True
True

进程已结束,退出代码0
```

3) hasattr(object, name): Return True if the name string is the name of one of the properties of the object object, False otherwise.

4) getattr(object, name): Return the value of the named property of the object. name must be a string. If the string name is one of the properties of the object, the value of that property is returned.

5) setattr(object, name, value): The string name refers to an existing property or a new property of the object. As long as the setattr operation is allowed by the object, the function will assign the value to the attribute.

6) delattr(object, name): The string name must be the name of an property of the object. If the deleting property is allowed by object, the delattr function will delete the specified name property.

7) **import(name)**: import modules dynamically

8) callable(object): If object can be called, True will be returned, otherwise False.

```
# hasattr,getattr,setattr delatr function
class A:
    def __init__(self):
        self.name ="python"
        self.age = 10
        self.sex = "male"
myclass= A()
print(hasattr(myclass,"name"))
print(getattr(myclass,"age"))
setattr(myclass,"num",12345)
print(vars(myclass))
delattr(myclass,"name")
print(vars(myclass))
# __import__ function
np = __import__("numpy")
print(np._version)
# callable function
print(callable(A))
```

```
/usr/bin/python3.8 /home/lin/PycharmProjects/pythonProject/test.py
True
10
{'name': 'python', 'age': 10, 'sex': 'male', 'num': 12345}
{'age': 10, 'sex': 'male', 'num': 12345}
<module 'numpy._version' from '/home/lin/.local/lib/python3.8/site-packages/numpy/_version.py'>
True
```

2.9. Variable Operation

1) globals(): Return a dictionary composed of global variables and their values in the scope

2) locals(): Return a dictionary consisting of local variables and their values in the current scope.

```
# globals function
print(globals())
```

```
/usr/bin/python3.8 /home/lin/PycharmProjects/pythonProject/test.py
{'__name__': '__main__', '__doc__': None, '__package__': None, '__loader__': <_frozen_importlib_external.SourceFil
```

```
# celocals function
def fune():
    local_varl =123
    ocal_var2 = "python"
    print(locals())
fune()
```

```
/usr/bin/python3.8 /home/lin/PycharmProjects/pythonProject/test.py
{'local_varl': 123, 'ocal_var2': 'python'}
```

2.10 Human-computer Operation

1) print(*objects, end='\n', file=sys.stdout): Print the contents of objects to the text stream specified by file, and end with end.

2) input(): read the value input by user

3) open(file, mode='r'): Open the file and return the corresponding file object, mode corresponds to the operation of reading or writing to the file. If the

file cannot be opened, the program will throw OSError.

```
# print function
print("python")
print(124)
# input function
input("Please enter a value:")
# open function
with open("a.txt",'w') as fw:
    fw.write("python")
    fw.write("in")
with open("a.txt",'r') as fr:
    print(fr.readlines())
```

```
python
124
Please enter a value:123
['pythonin']
```

2.11 Compilation Operation

1) compile(source,, mode): The function compiles the source into code which is then executed by the exec function or the eval function. The mode parameter indicates the mode adopted in compiling the code.

2) exec(object): The exec function supports dynamic execution of Python code, where object must be a string or a code object. If it is a string, the string will be parsed into a Python statement to execute, and if the two u fruit is coded, it will be executed directly.

3) eval(expression): The eval function evaluates the string expression as a valid expression and returns the result. The eval function can only evaluate a single expression, not complex code logic or assignment operation.

```
# complie,exec function
str_code = "print([i**2 for i in range (5) ])"
compilel = compile(str_code, '', mode="exec")
exec(compilel)
# eval function
print(eval("12*34+4"))
print(eval("float (200)"))
```

```
/usr/bin/python3.8 /home/lin/PycharmProjects/pythonProject/test.py
[0, 1, 4, 9, 16]
412
200.0
```

2.12 Decorator Function

1) classmethod(): It is used to to encapsulate a method into a class method, and the method modified by the decorator can be called without creating a class object.

2) staticmethod(): The staticmethod method is to convert the method in the class into a static method. The static method does not accept implicit parameters, and the static method can also be called without creating a class object.

3) property(): As a decorator, property can convert class methods into class properties for use.

```python
class MyClass(object):
    district = "BeiJing"

    def __init__(self, firstname="Joe", lastname="Harris"):
        self.firstname = firstname
        self.lastname = lastname
        self.age = 20

    # property,classmethod,staticmethod函数
    @property
    def fullName(self):
        print("ful1Name: ().  1)".format(self.firstname, self.lastname))

    @classmethod
    def showDistrict(cls):
        print("I an in ".format(cls.district))

    @staticmethod
    def showStaticmethod():
        print("this is showStaticmethod")


MyClass("hah", "lala").fullName
MyClass.showDistrict()
MyClass.showStaticmethod()
```

```
/usr/bin/python3.8 /home/lin/PycharmProjects/pythonProject/test.py
ful1Name: ().  1)
I an in
this is showStaticmethod
```

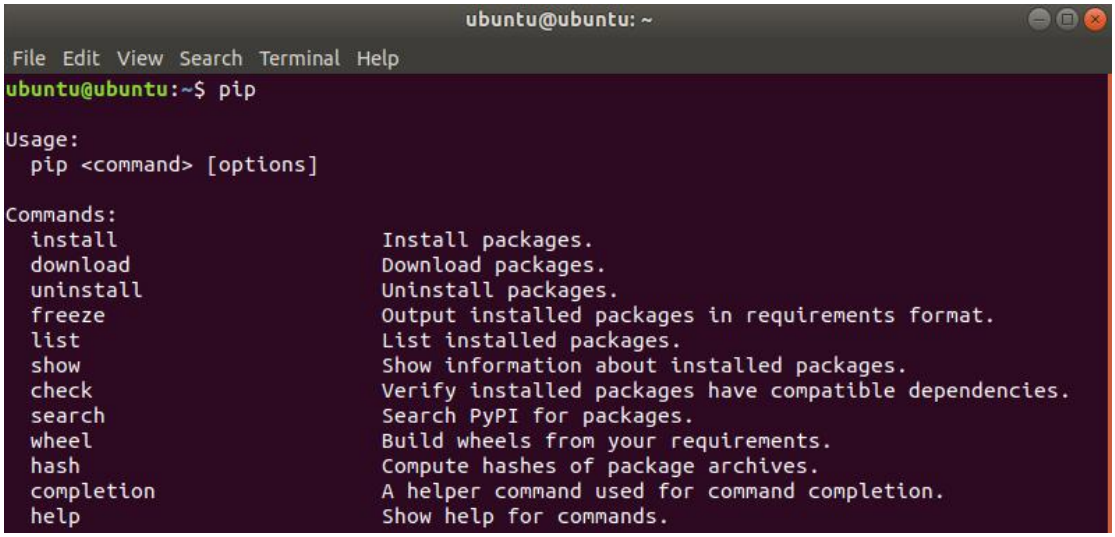
Lesson 12 Usage of Python PiP

1. Brief Introduction

pip is a management tool for Python package offering searching, downloading, installing and uninstalling function. Most of packages are built-in or are installed by pip. Python 2.7.9 +, Python 3.4+ and other versions above come with pip tool.

2. Download and Install

Open command line terminal, and then input “pip”. When the following message pops up, pip has been installed already.

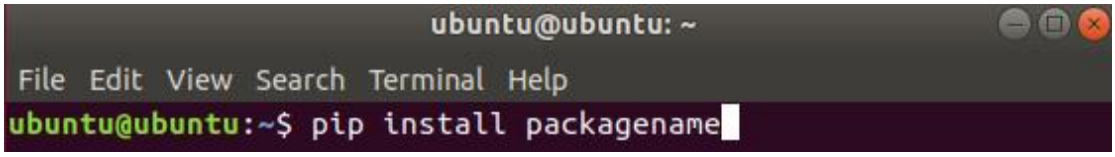


If pip is not installed, input command “**sudo apt install python-pip**” to install. After the command is executed, you will be required to input the password, and then select “y” to continue installation.

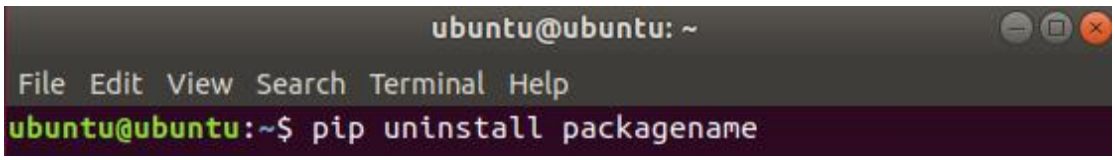
```
ubuntu@ubuntu: ~
File  Edit  View  Search  Terminal  Help
ubuntu@ubuntu:~$ sudo apt install python-pip
[sudo] password for ubuntu:
Reading package lists... Done
Building dependency tree
Reading state information... Done
The following additional packages will be installed:
  build-essential dpkg-dev fakeroot g++ g++-7 libalgorithm-diff-perl libalgorithm-diff-xs-per
l libalgorithm-merge-perl libexpat1-dev libfakeroot libpython-all-dev libpython-dev
  libpython-stdlib libpython2.7-dev libstdc++-7-dev make python python-all python-all-dev pyt
hon-asn1crypto python-cffi-backend python-crypto python-cryptography python-dbus
  python-dev python-enum34 python-gi python-idna python-ipaddress python-keyring python-keyri
ngs.alt python-minimal python-pip-whl python-pkg-resources python-secretstorage
  python-setuptools python-six python-wheel python-xdg python2.7 python2.7-dev python2.7-mini
mal
Suggested packages:
  debian-keyring g++-multilib g++-7-multilib gcc-7-doc libstdc++6-7-dbg libstdc++-7-doc make-
doc python-doc python-tk python-crypto-doc python-cryptography-doc
  python-cryptography-vectors python-dbus-dbg python-dbus-doc python-enum34-doc python-gi-cai
ro libkf5wallet-bin gir1.2-gnomekeyring-1.0 python-fs python-gdata python-keyczar
  python-secretstorage-doc python-setuptools-doc python2.7-doc binfmt-support
The following NEW packages will be installed:
  build-essential dpkg-dev fakeroot g++ g++-7 libalgorithm-diff-perl libalgorithm-diff-xs-per
l libalgorithm-merge-perl libexpat1-dev libfakeroot libpython-all-dev libpython-dev
  libpython-stdlib libpython2.7-dev libstdc++-7-dev make python python-all python-all-dev pyt
hon-asn1crypto python-cffi-backend python-crypto python-cryptography python-dbus
  python-dev python-enum34 python-gi python-idna python-ipaddress python-keyring python-keyri
ngs.alt python-minimal python-pip python-pip-whl python-pkg-resources
  python-secretstorage python-setuptools python-six python-wheel python-xdg python2.7 python2
.7-dev python2.7-minimal
0 upgraded, 43 newly installed, 0 to remove and 0 not upgraded.
Need to get 45.9 MB of archives.
After this operation, 104 MB of additional disk space will be used.
Do you want to continue? [Y/n] y
```

3. Common Command

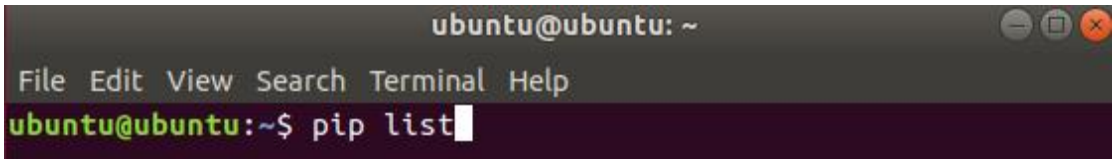
1) install: Install the command package, and the complete command is "**pip install packagename**"



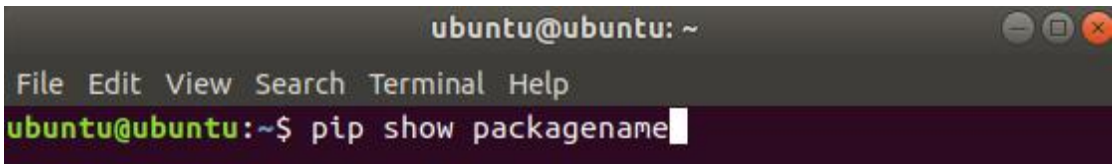
2) uninstall: Uninstall the designated package, and the complete command is "**pip uninstall packagename**"



3) list: List the installed package, and the complete command is "**pip list**"



4) show: View the information about the installed packages, and the complete command is “**pip show packagename**”.



Lesson 13 Python Numpy Basic Operation

1. Brief Introduction

NumPy (Numerical Python) is an extension library of Python, which supports a large number of dimensional array and matrix operations, and also provides a large number of mathematical function libraries for array operations.

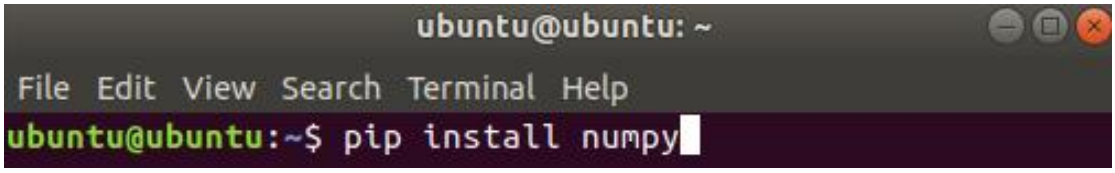
1) In some ways, Numpy arrays are greatly similar to built-in lists in Python. It is an essential tool for Python data science.

2) A numpy one-dimensional array can be considered as a vector, and a two-dimensional array is a data frame (precisely corresponding to the matrix in R).

3) The biggest difference between numpy arrays and lists in python is that the type of the data contained in the former should be the same, but there is no restriction on the latter.

2. Numpy Importing

Numpy is a third-party library that needs to be installed first. Open the command line terminal and enter the command to install "pip install numpy".



Then we need to import it in python with "import numpy". We can use "import numpy as np" to take "np" as an alias.

3. Create Array

1) np.array(): Create directly

```
# one-dimensional array
x1 = np.array([1,2,3])
x1.shape
# (3,) represents a one-dimensional array of length 3

# two-dimensional array
x2 = np.array([[1,3,5],[11,12,13]])
x2.shape
# (2,3) represent the 2-d matrix of two rows and three columns
```

3.1 Take Fixed Value

1) np.zeros(n, dtype=int): Create a one-dimensional array of integers of length n and all zeros

2) np.ones((a,b), dtype=float): Create a two-dimensional floating-point array with "**a**" row and "**b**" column, and all 1.

3) np.full((a,b),x, dtype=int/float): Create a two-dimensional array with "**a**" row and "**b**" column, and all **x**.

```
np.full((3,3),1.11)
# array([[1.11, 1.11, 1.11],
#        [1.11, 1.11, 1.11],
#        [1.11, 1.11, 1.11]])
```

3.2 Regular Distribution

1) np.arange(): arithmetic progression

```
In [144]: np.arange(10)
Out[144]: array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])

In [145]: np.arange(1,10,2)
Out[145]: array([1, 3, 5, 7, 9])
```

2) np.random.randint(): Random integer

```
In [148]: np.random.randint(0,100,10)
Out[148]: array([85, 48, 49, 69, 41, 35, 64, 95, 69, 94])

In [149]: np.random.randint(0,10,(3,3))
Out[149]:
array([[0, 1, 2],
       [4, 2, 0],
       [3, 2, 0]])
```

3) np.random.random(10) : A one-dimensional array with a length of 10 and distributed uniformly from 0 to 1.

4) np.random.normal(0, 1, c(2,3)): A two-dimensional array normally distributed with two rows and three columns, a mean of 0 and a standard deviation of 1.

4. Take Subset

4.1 Index Starts from 0

1) Similar to the way list takes subset in Python, the index of numpy array also starts from 0.

2) The index of negative can be seen as the n^{th} last value.

4.2 Take Subset of Single Value

View with the element subscript.

```
In [151]: x1 = np.arange(10)

In [152]: x1
Out[152]: array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])

In [153]: x1[9]
Out[153]: 9
```

4.3 Slice Takes Subset

1) For the one-dimensional array x1 of the length of 10, x1[0:5] represents the subset from index0 to index4, and the step size is 1 by default.

2) When a of [a:b:c] is 0, a can be omitted. When both a and b are omitted, it represents the complete set.

3) When the step size is negative, it represents take subset in reversed order.

```
In [162]: x1[1:7]
Out[162]: array([1, 2, 3, 4, 5, 6])

In [163]: x1[:5]
Out[163]: array([0, 1, 2, 3, 4])

In [164]: x1[2:8:2]
Out[164]: array([2, 4, 6])

In [165]: x1[9:5:-1]
Out[165]: array([9, 8, 7, 6])
```

4.4 Multiple Index of the List Take Subset

Take the subset in sequence according to the index.

```
In [38]: x1
Out[38]: array([2, 3, 6, 7, 0, 8, 2, 7, 6, 8])

In [39]: x1[[0,2,6]]
Out[39]: array([2, 6, 2])

In [40]: x2
Out[40]:
array([[11, 11,  2,  4],
       [ 0,  9,  9,  7],
       [ 2,  5,  0, 10]])

In [41]: x2[:,[1,2]]
Out[41]:
array([[11,  2],
       [ 9,  9],
       [ 5,  0]])
```

4.4 Logical Boolean Takes Subset

```
In [35]: x1
Out[35]: array([2, 3, 6, 7, 0, 8, 2, 7, 6, 8])

In [36]: x1[(x1 < 4) | (x1 > 7)]
Out[36]: array([2, 3, 0, 8, 2, 8])

In [37]: x1[~((x1 >= 4) & (x1 <= 7))]
Out[37]: array([2, 3, 0, 8, 2, 8])
```

4.5 Create Unrelated Subset Replicas

copy() can create unrelated subset replicas

```
In [185]: x2_sub = x2[:2, :3].copy()

In [186]: x2_sub[0,0] = 10000

In [187]: x2_sub
Out[187]:
array([[10000,  1000,     3],
       [   18,    14,     9]])

In [188]: x2
Out[188]:
array([[   18,  1000,     3,    17],
       [   18,    14,     9,     1],
       [    4,    10,    11,     8]])
```

5.Array Dimension Transforming

1) Transpose matrix with T()：Transform row and column

```
In [200]: x2
Out[200]:
array([[   18,  1000,     3,    17],
       [   18,    14,     9,     1],
       [    4,    10,    11,     8]])

In [201]: x2.T
Out[201]:
array([[   18,    18,     4],
       [1000,    14,    10],
       [    3,     9,    11],
       [   17,     1,     8]])
```

2) Transform dimension with reshaple(): transform the dimension according to your desire.

```
In [204]: x2
Out[204]:
array([[  18, 1000,    3,   17],
       [  18,   14,    9,    1],
       [   4,   10,   11,    8]])

In [205]: x2.reshape((2,6))
Out[205]:
array([[  18, 1000,    3,   17,   18,   14],
       [   9,    1,    4,   10,   11,    8]])

In [206]: x2.reshape((1,12))
Out[206]:
array([[  18, 1000,    3,   17,   18,   14,    9,    1,    4,   10,   11,
          8]])
```

6. Arrays Merging and Splitting

6.1 Array Merging

1) concatenate() : Connect two one-dimensional array or merge two two-dimensional arrays according to row and column.

```
In [207]: x1_1 = np.random.randint(10, size = 3)

In [208]: x1_2 = np.random.randint(20, size = 5)

In [209]: np.concatenate([x1_1,x1_2])
Out[209]: array([ 2,  3,  0,  0,  0,  6, 19, 14])

In [210]: np.concatenate([x1_1,x1_2,x1_2])
Out[210]: array([ 2,  3,  0,  0,  0,  6, 19, 14,  0,  0,  6, 19, 14])
```

2) vstack(): Merge two arrays vertically, as long as these two arrays have the same number of columns.

```
In [222]: np.vstack([x2_1, x2_2])
Out[222]:
array([[ 8,  0,  8],
       [ 5,  9,  0],
       [60, 61, 83],
       [33, 32, 70]])
```

3) hstack(): Merge two arrays horizontally, as long as these two arrays

have the same number of rows.

```
In [223]: np.hstack([x2_1, x2_2])
Out[223]:
array([[ 8,  0,  8, 60, 61, 83],
       [ 5,  9,  0, 33, 32, 70]])
```

6.2 Split Array

1) split(): Split one-dimensional array. Where to split will be designated by the index of the parameter. When the function returns multiple objects, we need to set the corresponding numbers of object names in front of the equal sign. When the function result returns multiple objects, set the corresponding number of object names before the equal sign

```
In [224]: x1 = np.random.randint(10, size = 10)

In [225]: x1_1, x1_2, x1_3 = np.split(x1, [2,5,])

In [226]: x1_1
Out[226]: array([5, 7])

In [227]: x1_2
Out[227]: array([8, 8, 9])

In [228]: x1_3
Out[228]: array([2, 8, 6, 6, 9])
```

2) vsplit(): Split a two-dimensional array horizontally. And hsplit() splits a two-dimensional array vertically.

```
In [229]: x2 = np.random.randint(12, size = (3,4))

In [230]: x2_up, x2_down = np.vsplit(x2,[1])

In [231]: x2_up
Out[231]: array([[1, 6, 8, 8]])

In [232]: x2_down
Out[232]:
array([[ 3,  2,  3, 10],
       [ 6,  3,  6,  5]])

In [233]: x2_left, x2_right = np.hsplit(x2,[1])

In [234]: x2_left
Out[234]:
array([[1],
       [3],
       [6]])

In [235]: x2_right
Out[235]:
array([[ 6,  8,  8],
       [ 2,  3, 10],
       [ 3,  6,  5]])
```